

TT-Ascalon

Physical Design Reference Guide

Revision 2.1
April 30, 2025

© 2025 Tenstorrent Holdings Inc. and its subsidiaries.

The information contained herein is for informational purposes only and is subject to change without notice. While every precaution has been taken in the preparation of this document, it may contain technical inaccuracies, omissions and typographical errors, and Tenstorrent is under no obligation to update or otherwise correct this information. Tenstorrent, Inc. makes no representations or warranties with respect to the accuracy or completeness of the contents of this document, and assumes no liability of any kind, including the implied warranties of noninfringement, merchantability or fitness for particular purposes, with respect to the operation or use of Tenstorrent products described herein. No license, including implied or arising by estoppel, to any intellectual property rights is granted by this document. Terms and limitations applicable to the purchase or use of Tenstorrent products are as set forth in a signed agreement between the parties or in the Tenstorrent Standard Terms and Conditions of Sale.

TRADEMARKS

A list of Tenstorrent's trademarks can be found at <https://tenstorrent.com/trademarks>. Other product names and links to external sites used in this publication are for identification purposes only and may be trademarks of their respective companies.

*Links to third party sites are provided for convenience and unless explicitly stated, Tenstorrent is not responsible for the contents of such linked sites and no endorsement is implied.

Table of Contents

Table of Contents	2
Revision History	3
Introduction	4
Physical Hierarchies	5
Synthesis Guideline	6
PNR Guideline	8
Standard Cell	9
Memories	11
tt_rv_fe Block	11
tt_ls_ls1 Block	11
tt_ls_ls2 Block	11
tt_sc_slice_top Block	11
tt_cluster Block	12
Constraints	13
Logic Equivalence Check	15
CPU Core	16
Pin Placement	16
CPU Core Block Details	18
tt_rv_ls	18
tt_ls_ls1 Recommendations	18
tt_ls_ls2 Recommendations	20
tt_rv_fe	21
tt_rv_fe Recommendations	21
tt_rv_mc	23
tt_mc_mc1	24
tt_mc_mc2	24
tt_rv_ms	25
Shared Cache	27
tt_sc_slice_top	27
sc_slice_top	28
tt_sc_partition	29
sc_partition Recommendations	31
Cluster	33
Cluster logic placement	33
sc0 hierarchy	34
Ring Networks	36
Clock Distribution	38

Revision History

Revision	Date	Notes
2.0	Apr, 2025	Second release
2.0	Mar, 2025	Updated formatting, proofreading
1.3	Nov, 2024	Proofreading
1.2	Oct, 2024	Updated block names, scripts
1.1	Sep, 2024	Updated formatting and branding.
1.0	Aug, 2024	First release

Introduction

This guide describes running synthesis, P&R, and logic equivalency checks for the TT-Ascalon Cluster IP. The top-level TT-Ascalon Cluster design is an 8-wide RISC-V CPU. All cores share the 12MB L2 cache.

TENSTORRENT CONFIDENTIAL
FOR CORELAB

Physical Hierarchies

Figure 1 defines the physical hierarchies from which the synthesis and PNR flows are run.

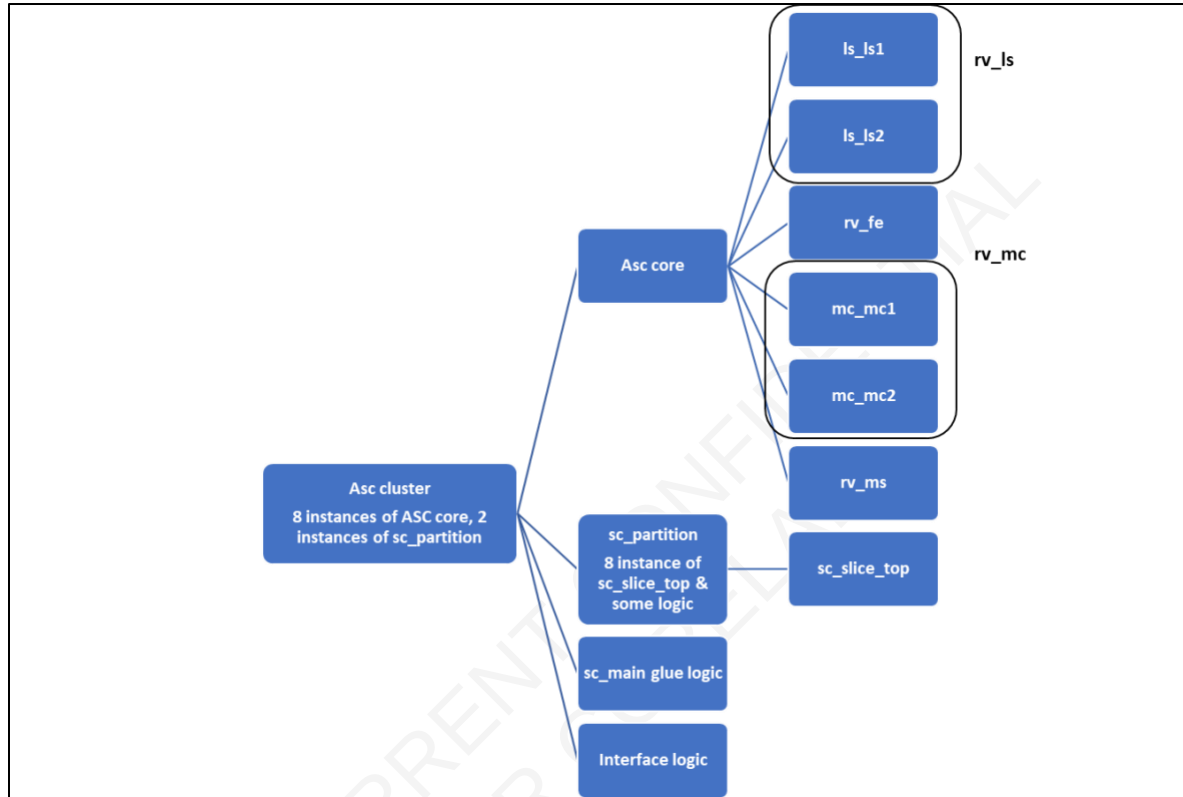


Figure 1 - TT-Ascalon physical hierarchies

Synthesis Guideline

Tenstorrent recommends standard synthesis settings with Unloaded and Constant Flop Optimization enabled. Physical synthesis is recommended for good QOR.

Set the `$ROOT` environment variable to `<package untar directory>/tt_ascalon`. The `rtl_list` file uses this environment variable to access the `rtl` files. The synthesis definitions should be `SYNTHESIS, NO_LIBCELL, TT_BEHAVIORAL_MEM`. This assumes:

- You do not have the memory models updated in the `$ROOT/source/integration/` memory files.
 - The technology-specific libcells in `$ROOT/source/primitives/tt_libcell*.sv` are not updated. The `$ROOT/source/primitives/tt_libcells*.sv` files contain the library cells used in the design. The current TT-Ascalon only uses the `tt_clkgater`. Update each `$ROOT/source/primitives/tt_libcells*.sv` file as needed for custom technology nodes.
- If you do update these two files, then remove the `NO_LIBCELL` and `TT_BEHAVIORAL_MEM` from your synthesis definitions and then rerun.

The top module names for the different physical hierarchies are `tt_ls_ls1`, `tt_ls_ls2`, `tt_rv_fe`, `tt_mc_mc1`, `tt_mc_mc2`, `tt_rv_ms`, `tt_sc_slice_top`, `tt_sc_partition`, `tt_core` and `tt_cluster`.

- Use the correct physical hierarchical module name when elaborating to avoid errors during the elaboration.
- When running synthesis at `sc_partition` or at the cluster level, ensure that the lower-level blocks are picked from `libs/LEFs` and that you are not synthesizing them.

The `rtl_list` file for all the blocks is in the `$ROOT/cluster/physical_design` directory. You can execute the following synthesis commands elaborate the design at each level of the hierarchy:

```
make pd BLOCK=<block> TOOL=dc
Ex: make pd BLOCK=tt_ls_ls1 TOOL=dc
```

TENSTORRENT CONFIDENTIAL
FOR CORELAB

PNR Guideline

The floorplan and pin placement details provided for all the blocks are from a different technology node. It is imperative to understand that they may not work, as they are for a different technology node. The floorplan and placements are provided as a guideline for building the design. Tenstorrent recommends enabling Multibit for power optimization.

TENSTORRENT CONFIDENTIAL
FOR CORELAB

Standard Cell

Instantiate standard cells from target process node/library in the verilog wrappers for standard cells. Refer to the Integration Guide for more details.

File Name	Functionality
tt_libcell_aoi22_quad.sv	Quad AOI22 gate
tt_libcell_aoi22.sv	AOI22 gate
tt_libcell_clkand2.sv	AND gate with clock input
tt_libcell_clkbuf.sv	Clock buffer
tt_libcell_clkgate.sv	Clock gate
tt_libcell_clkinv.sv	Clock inverter
tt_libcell_clkmux2.sv	2-to-1 clock multiplexer
tt_libcell_clknand2.sv	NAND gate with clock input
tt_libcell_clkor2.sv	OR gate with clock input
tt_libcell_clkxor2.sv	XOR gate with clock input
tt_libcell_dffrxq.sv	D flip-flop with active-low reset and Q output
tt_libcell_dffsrqxq.sv	D flip-flop with active-low set/reset and Q output
tt_libcell_dff.sv	D flip-flop
tt_libcell_dffsxq.sv	D flip-flop with active-low set and Q output
tt_libcell_fulladder.sv	Full adder
tt_libcell_halfadder.sv	Half adder
tt_libcell_lhcnqd4.sv	Latch with clear and Q output
tt_libcell_metastab_hardened_dffr.sv	Metastability-hardened D flip-flop with active-low reset
tt_libcell_metastab_hardened_dff.sv	Metastability-hardened D flip-flop
tt_libcell_nd2.sv	2-input NAND gate
tt_libcell_nd3.sv	3-input NAND gate
tt_libcell_nd4.sv	4-input NAND gate
tt_libcell_or2.sv	2-input OR gate
tt_libcell_stdbuf.sv	Standard buffer
tt_libcell_stdinv.sv	Standard inverter
tt_libcell_stdmux2.sv	2-to-1 multiplexer
tt_libcell_stdmux3.sv	3-to-1 multiplexer
tt_libcell_stdand2.sv	2-input AND gate
tt_libcell_stdnand2.sv	2-input NAND gate
tt_libcell_stdor2.sv	2-input OR gate
tt_libcell_sync2r_np.sv	2-stage synchronizer with active-low reset (non-pipelined)
tt_libcell_sync2r.sv	2-stage synchronizer with active-low reset
tt_libcell_sync2s.sv	2-stage synchronizer with active-low set
tt_libcell_sync2.sv	2-stage synchronizer

tt_libcell_sync3r.sv	3-stage synchronizer with active-low reset
tt_libcell_sync3s.sv	3-stage synchronizer with active-low set
tt_libcell_sync3.sv	3-stage synchronizer
tt_libcell_sync4r.sv	4-stage synchronizer with active-low reset
tt_libcell_sync4s.sv	4-stage synchronizer with active-low set
tt_libcell_sync4.sv	4-stage synchronizer
tt_libcell_xor2.sv	2-input XOR gate

Table 1. Standard cell modules

Memories

Tables 2 - 6 define the memories used in every block and their corresponding type and instance counts.

tt_rv_fe Block

Memory Name	Count	Memory Instance Name	Memory Type
icache	32	mem_cell_1rw_0r_0w_256x78_1	Single Port RF
itag	8	mem_cell_1rw_0r_0w_64x92_0	Single Port RF
cic	16	mem_cell_1rw_0r_0w_128x78_0	Single Port RF
ittage	2	mem_cell_1rw_0r_0w_128x78_1	Single Port RF
ittage	6	mem_cell_1rw_0r_0w_512x44_0	Single Port RF
ittage	4	mem_cell_1rw_0r_0w_128x88_1	Single Port RF
tage	8	mem_cell_1rw_0r_0w_256x50_1	Single Port RF
tage	12	mem_cell_1rw_0r_0w_512x44_0	Single Port RF
tlb_ram	1	mem_cell_1rw_0r_0w_64x96_0	Single Port RF
bim	2	mem_cell_1rw_0r_0w_256x50_1	Single Port RF

Table 2. rv_fe memory

tt_ls_ls1 Block

Memory Name	Count	Memory Instance Name	Memory Type
DTag	48	mem_cell_1rw_0r_0w_256x50_0	Single Port RF
DCache	128	mem_cell_1rw_0r_0w_256x78_1	Single Port RF

Table 3. tt_ls_ls1 memory

tt_ls_ls2 Block

Memory Name	Count	Memory Instance Name	Memory Type
Prefetcher	6	mem_cell_1rw_0r_0w_256x132_0	Single Port RF
L2TlbRam	12	mem_cell_1rw_0r_0w_256x72_0	Single Port RF
L2TlbCam	12	mem_cell_1rw_0r_0w_256x96_0	Single Port RF

Table 4. tt_ls_ls2 memory

tt_sc_slice_top Block

Memory Name	Count	Memory Instance Name	Memory Type
sdata	48	mem_cell_1rw_0r_0w_2048x72_0	Single Port SRAM
sctag	12	mem_cell_1rw_0r_0w_512x96_0	Single Port RF
snftag	8	mem_cell_0rw_1r_1w_256x96_0	Two Port RF
snfstate	4	mem_cell_0rw_1r_1w_256x96_0	Two Port RF
screplace/scstate	4	mem_cell_0rw_1r_1w_256x128_0	Two Port RF

Table 5. sc_slice_top memory

tt_cluster Block

Memory Name	Count	Memory Instance Name	Memory Type
Trace RAM	8	mem_cell_1rw_0r_0w_512x64_0	Single Port RF
SMC RAM	4	mem_cell_1rw_0r_0w_2048x72_1	Single Port SRAM

Table 6. Cluster memory

The memory wrappers are in `$ROOT/source/primitives/tt_mem_gen.sv` and use the naming convention `mem_cell_{read/write ports}rw_{read_ports}r_{write_ports}w_{width}x{height}_{some_number}`

In the memory instance names:

- **1rw** refers to a single port memory.
- **1r_1w** refers to two port memories.

Either:

- Update the memory instance names in this file as needed.
- Update the separate memory files provided in the `$ROOT/source/integration` directories.

Add any required macro constraints to ensure correct functionality/timing, such as case analysis from the data sheet.

Constraints

The `$ROOT/cluster/physical_design/constraints` directory provides constraints for each physical hierarchy. The core clock (`clk`) is the critical clock in this design. Update the frequency of this clock in all SDC files.

Clock Table:

Clock	Frequency	Blocks
Core clock (<code>clk</code>)	Technology dependent	<code>tt_ls_ls1</code> , <code>tt_ls_ls2</code> , <code>tt_rv_fe</code> , <code>tt_mc_mc1</code> , <code>tt_mc_mc2</code> , <code>tt_rv_ms</code> , <code>tt_sc_slice_top</code> , <code>tt_sc_partition</code> , <code>tt_core</code> , <code>tt_cluster</code>
Fabric Clock (<code>fb_clk</code>)	Technology dependent	<code>tt_cluster</code>
SOC clock (<code>soc_clk</code>)	400 MHz Fixed Frequency	<code>tt_cluster</code>
Reference Clock (<code>rf_clk</code>)	100 MHz Fixed Frequency	<code>tt_cluster</code>
Tap Clock (<code>tck</code>)	200 MHz Fixed Frequency	<code>tt_cluster</code>

Table 7. Cluster clocks

The default IO delay expectations are coded in the constraints file. Some pins need to be customized based on the floorplan and logic distribution between blocks.

- Output pins on all blocks consume 30% of the period.
- Input pins on all blocks consume 70% of the period.

Update additional constraints like `set_driving_cells` with technology-specific information. Add `set_dont_touch` on the `dont` cells as they are expected to be preserved in the design.

TENSTORRENT CONFIDENTIAL
FOR CORELAB

Logic Equivalence Check

Tenstorrent used Synopsys® Formality® to test all blocks for basic elaboration. Here is a sample Formality script to read and elaborate on the design:

```
make pd BLOCK=<block> TOOL=fm  
Ex: pd BLOCK=tt_ls_ls1 TOOL=fm
```

TENSTORRENT CONFIDENTIAL
FOR CORELAB

CPU Core

Figure 2 shows the reference CPU core floorplan. This floorplan is abutted, and there are no channels present between any partitions. Feedthrough ports route signals between non-adjacent blocks, top level ports, and partition pins.

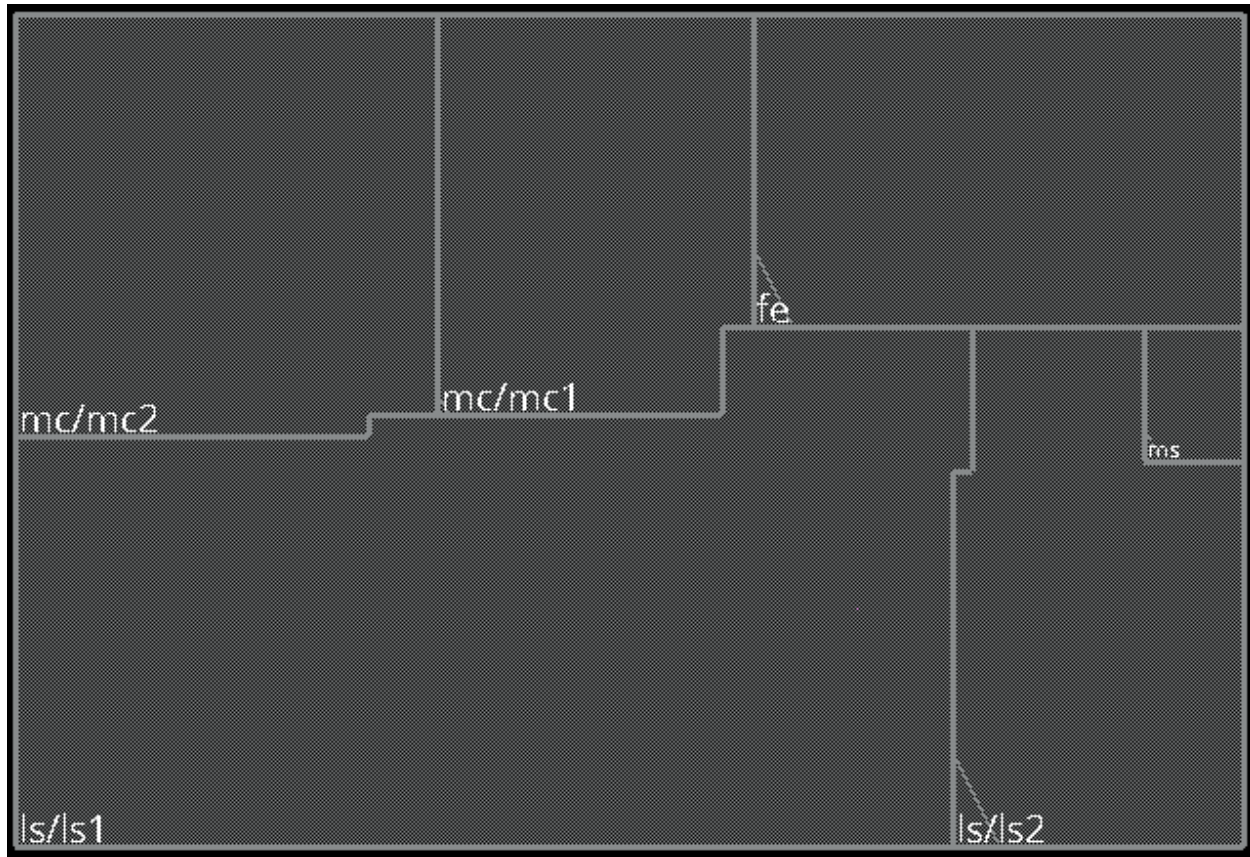


Figure 2 - CPU Core Floorplan

Pin Placement

Figure 3 shows the main buses between the partitions and the top level I/Os. The general pin naming convention is **<Driving Partition>_<Receiving Partition>_<Signal Name>**.

The exceptions to the naming rule are for:

- Connections between **ls1** and **mc_mc1/mc_mc2**, which use the convention **ls_mc_<Signal Name>** or **mc_ls_<Signal Name>** depending on direction.
- Connections between **ls1** and **ls2**, which use the convention **<Driving Module>_<Receiving Module>_<Signal Name>**, where **Module** is the internal module hierarchy name in **ls1** and **ls2**.

Use the given instance name given when placing the pins. For example, place pins from **ls1** to **mc_mc1** by taking pins named **mc_mc1/ls_mc_*** and placing them on the **mc_mc1/ls** boundary. The floorplanning tool can then place the corresponding pins on **ls** to their matching **mc_mc1** locations.

Figure 3 shows additional pins circled in yellow. These are either:

- Connections from one partition to multiple partitions (denoted with **<Driver>_ALL_<Signal Name>**).
- Connections between **rv_ms** and the other partitions that are routed through the feedthrough ports.

The floorplanning tool can auto-place these pins.

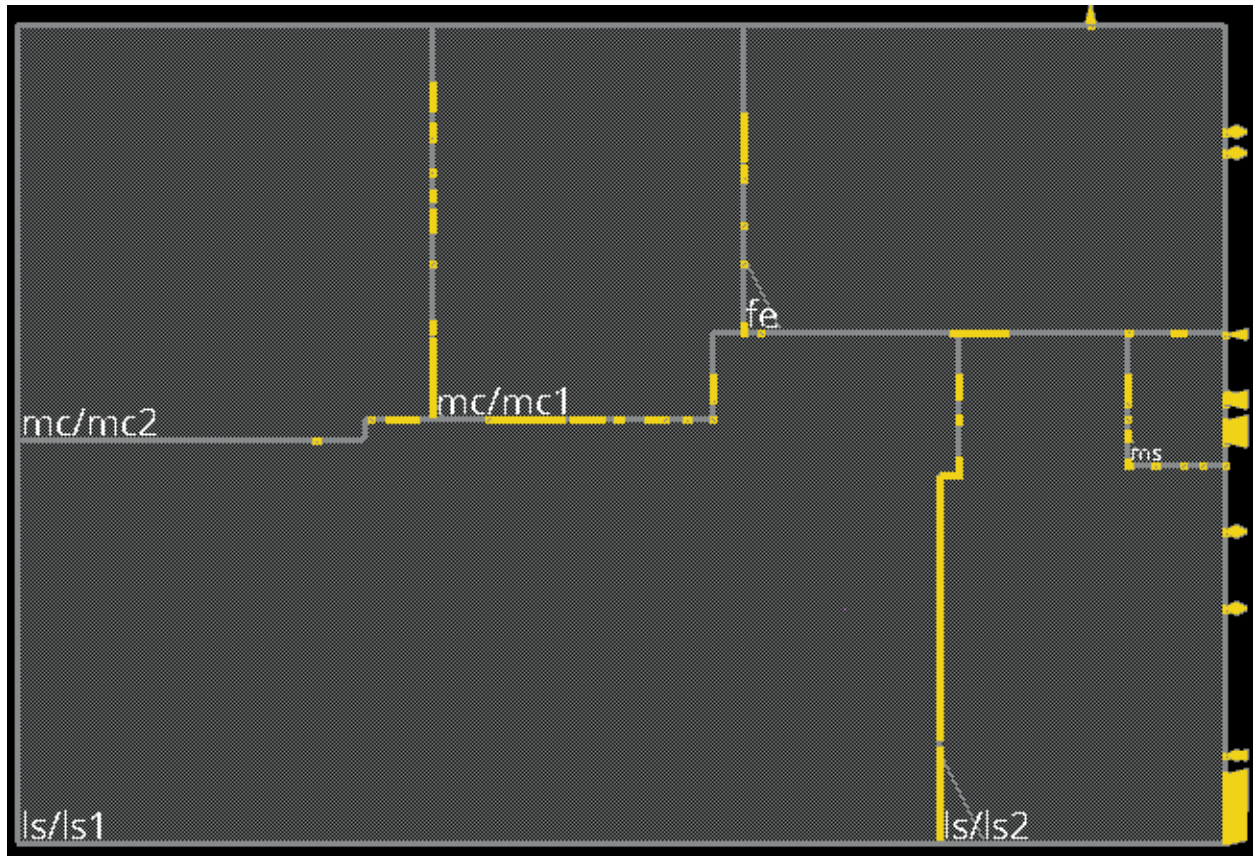


Figure 3 - CPU core pin locations

CPU Core Block Details

tt_rv_ls

tt_rv_ls is the Load/Store unit currently partitioned as **tt_ls_ls1** and **tt_ls_ls2** blocks for faster physical design turnaround times. The **tt_ls_ls1** (the larger block among the two **ls** partitions) consists of:

- The L1 data cache
- Tag macros
- Way predictor
- **L1TLBPrefet**
- Load/store queues
- Address generation pipe (**agp**) logic
- Arbitration (**arb**) logic.

The **tt_ls_ls2** block has L2 cache interface (**cif**), prefetcher logic (**pfc**), and L2TLB/PTW related logic (**mmu**).

Figure 4 shows the floorplan for the **tt_ls_ls1** and **tt_ls_ls2** blocks along with relevant macro and pin placement.



Figure 4 - tt_ls_ls1 and tt_ls_ls2 floorplans

tt_ls_ls1 Recommendations

The **tt_ls_ls1** tile contains 4-cycle load to use the architectural critical path:

1. **DF1** - Receive address register from **ms_mc1** (SrcA), compute the effective address, and set up to the data cache macros.
2. **DF2** - Read out of the data cache, then mux down the data per fold.
3. **DF3** - Final muxing between the fold, then mux in additional data sources.
4. **DF4** - Align data, then send it back to **mc_mc1** and **mc_mc2** blocks.

The first (**DF1**) cycle is the most critical and contains additional levels of logic. You may need to skew the clocks for the data cache and data tag macros to meet timing. Clocks are skewed for dcache by being pushed out in increments from the closest rightmost macros to the farthest leftmost macros, as shown in Figure 5. Figure 5 also highlights the above-mentioned 4-cycle load to use timing paths.

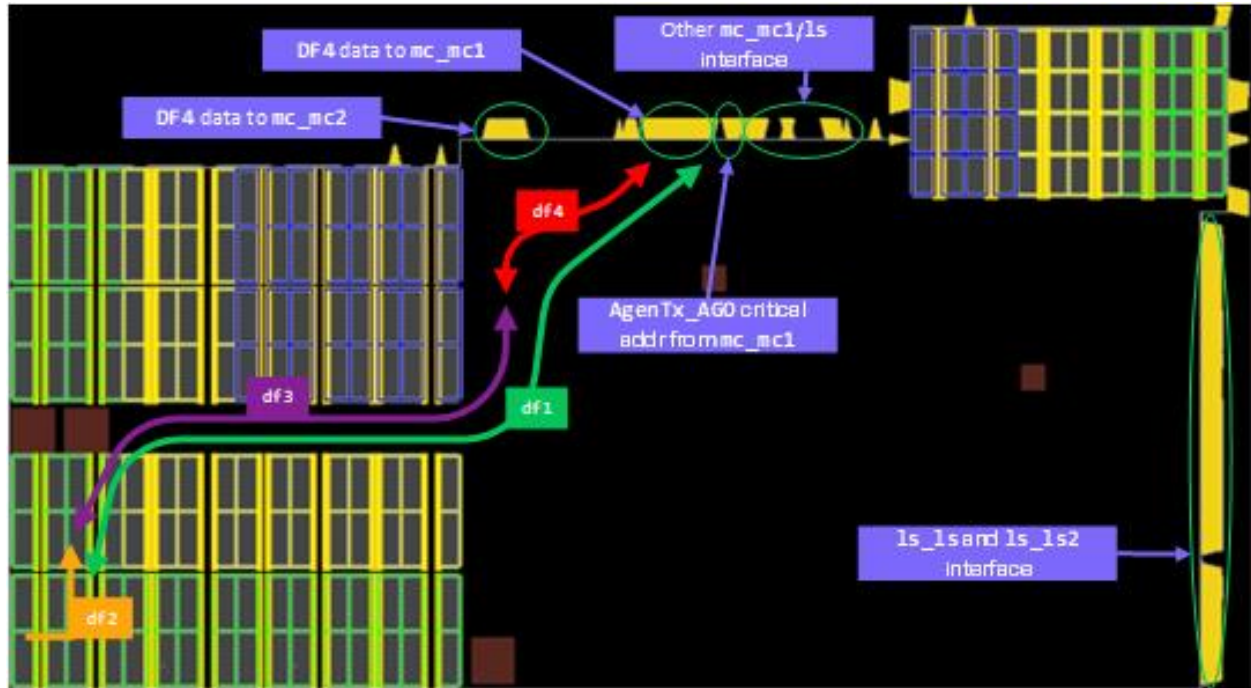


Figure 5 - 4-cycle load to use timing paths

Fig 6. highlights all level 1 hierarchies within the **tt_ls_ls1** block. Current design constraints include user skew on **dfp** and **dctag** as described above, plus:

- Regions for the **dfp/dcwayp**, **arb**, and **tlp** modules.
- **path_groups** to critical failing endpoints, such as the 4-cycle load to use paths.

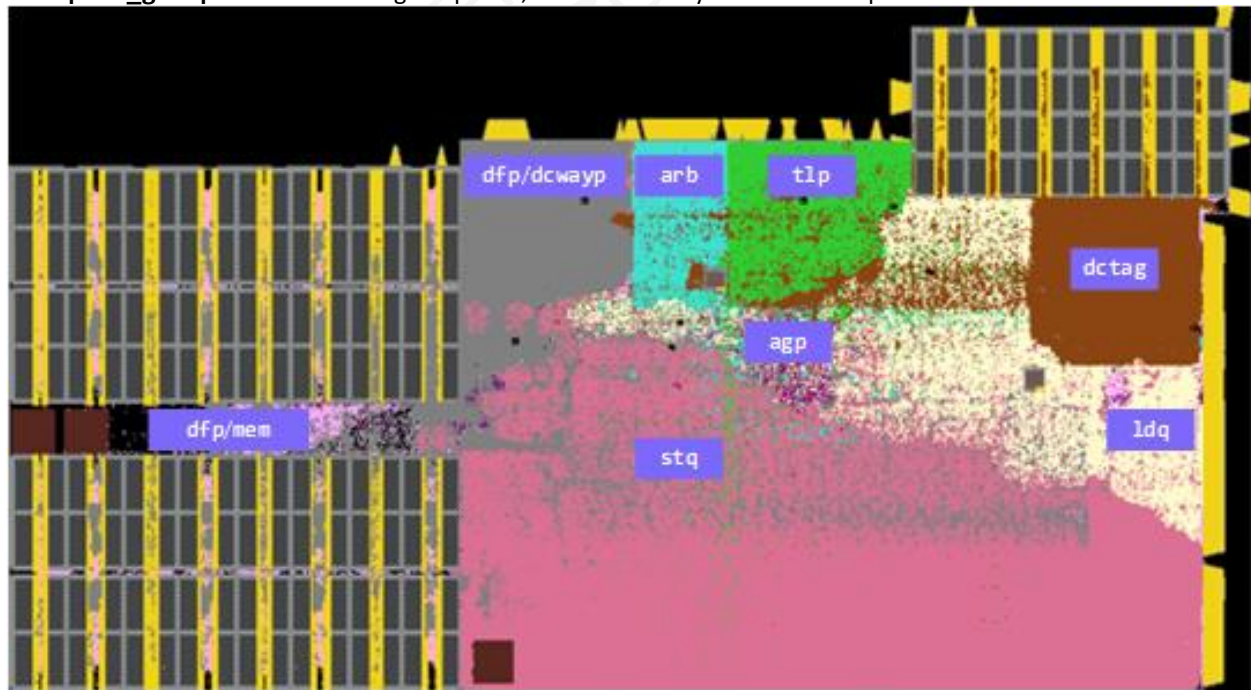


Figure 6 - ls1 module placement

tt_ls_ls2 Recommendations

The **tt_ls_ls2** tile consists of the **pfc**, **mmu**, **lp1**, and **cif** modules. **tt_ls_ls2** mostly interfaces with the **tt_ls_ls1** tile, **FE**, and shared cache in the cluster. Critical timing paths are interface paths with **tt_ls_ls1** and are addressed by carefully budgeting these IO paths.

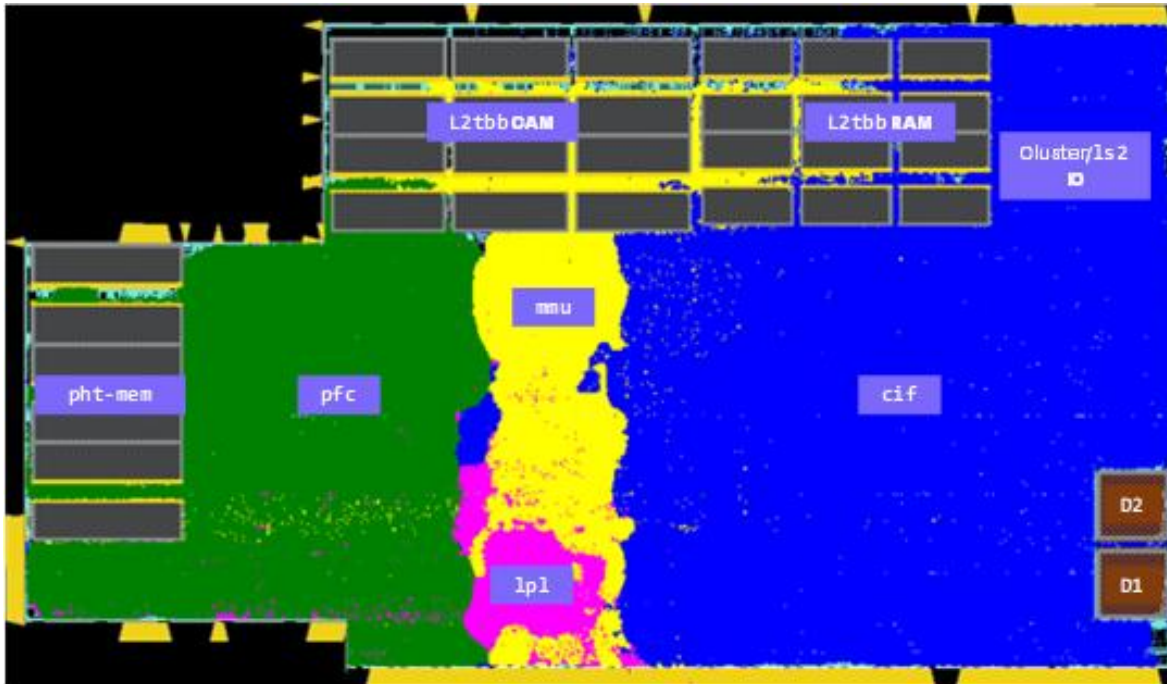


Figure 7 - tt_ls_ls2 module placement

tt_rv_fe

The **tt_rv_fe** block consists of the instruction cache, branch prediction, and decode/instruction dispatch logic. The tile is macro dominated.

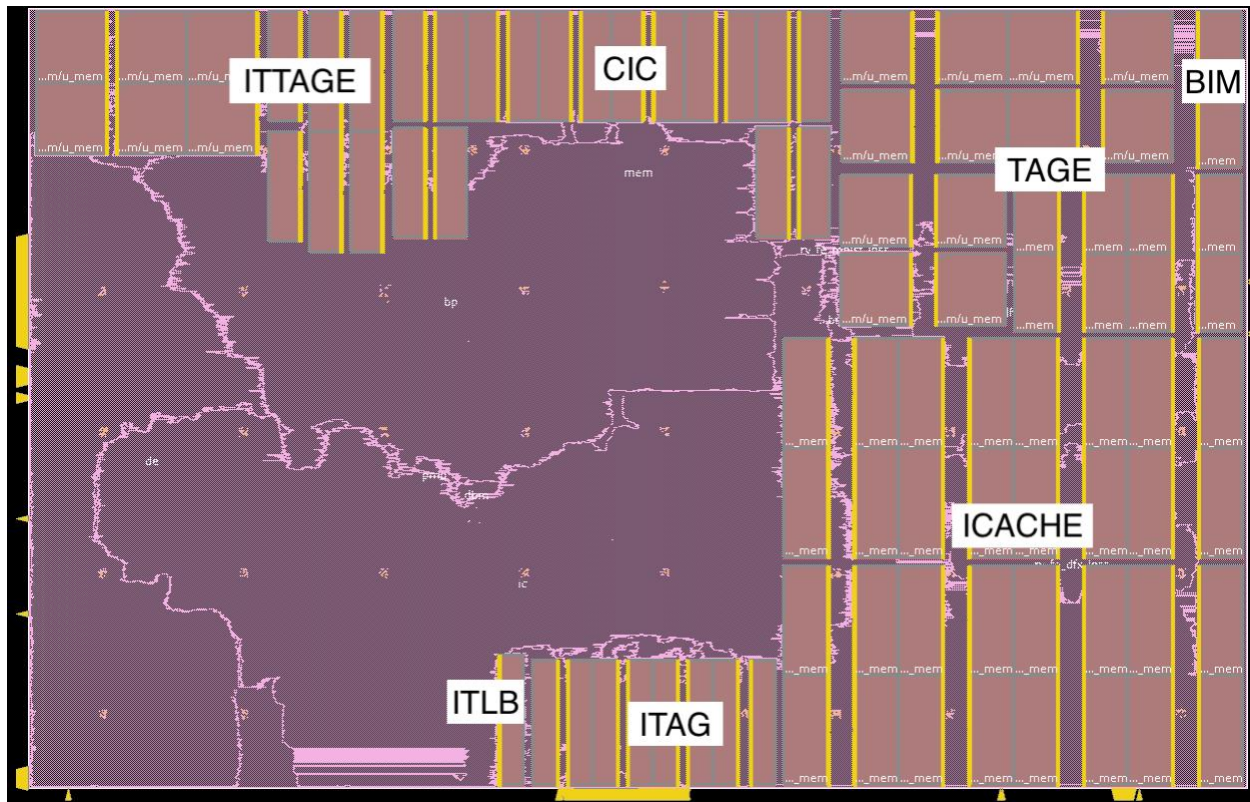


Figure 8 - tt_rv_fe floorplan

tt_rv_fe Recommendations

tt_rv_fe is a 6-stage pipeline design with three pipes working in parallel:

1. Control Prediction Pipe (**bp** hierarchy) - Prediction logic for fetching the next instruction.
2. Tag Logic (**ic** hierarchy) – to determine availability of required instructions in **Icache**.
3. Data Pipe (**ic** hierarchy) – access data from **Icache** and convert to **uops**.

There are many timing-critical macro paths because the initial data for preprocessing comes from macros in all three parallel pipes. Place macros according to logical hierarchy. The **Icache** macro is critical both on the output and data/chip enable sides. You can skew clocks based on macro timing. The Compressed Instruction Cache (**CIC**), **Tag**, and **ITTage** macros are critical on the output side.

Some of the major hierarchical blocks whose fanouts should be analyzed for possible placement optimizations include **bp_bst**, **bp_nfp**, **ic_ibuf**, **ic_ifbuf**, **ic_wp**, and **de**.

Recommended Floorplan and Major Module Placement

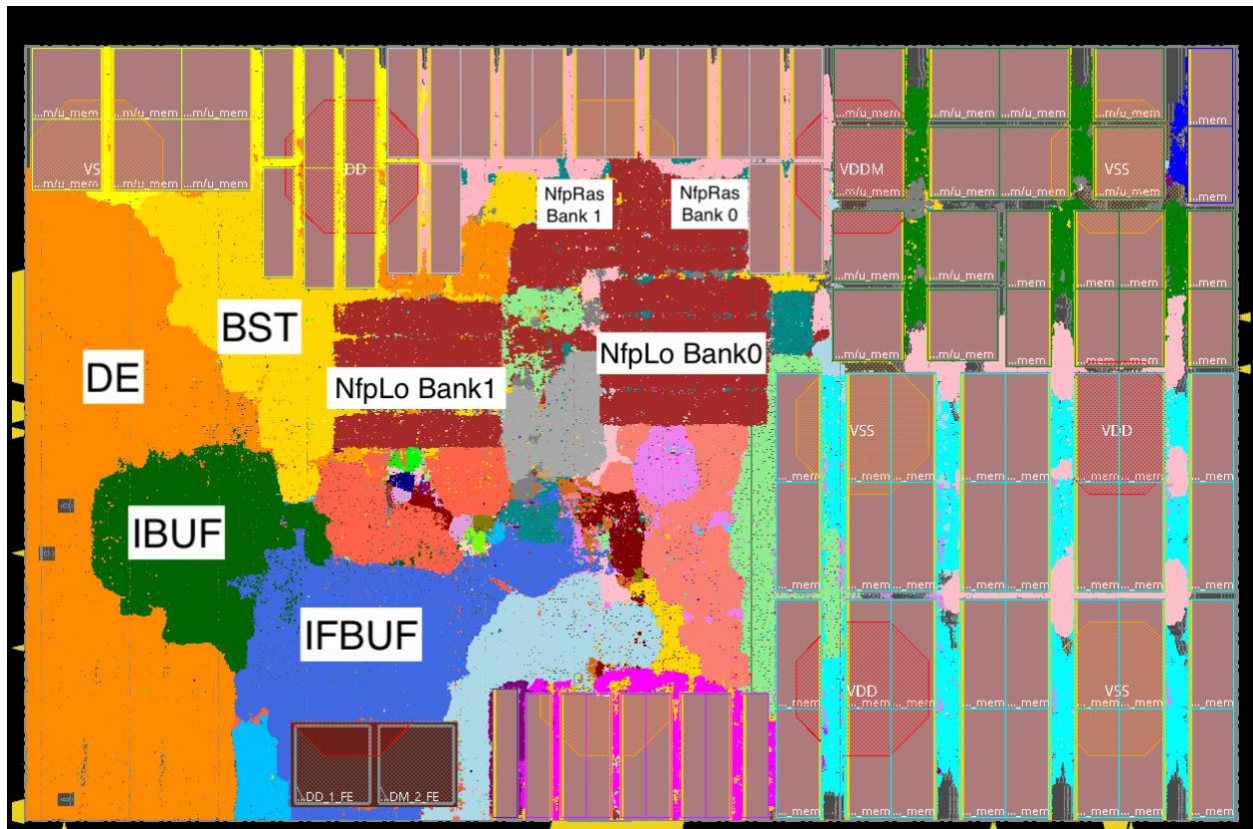


Figure 9 - tt_rv_fe module placement

Deeper Look Into DE Hierarchy

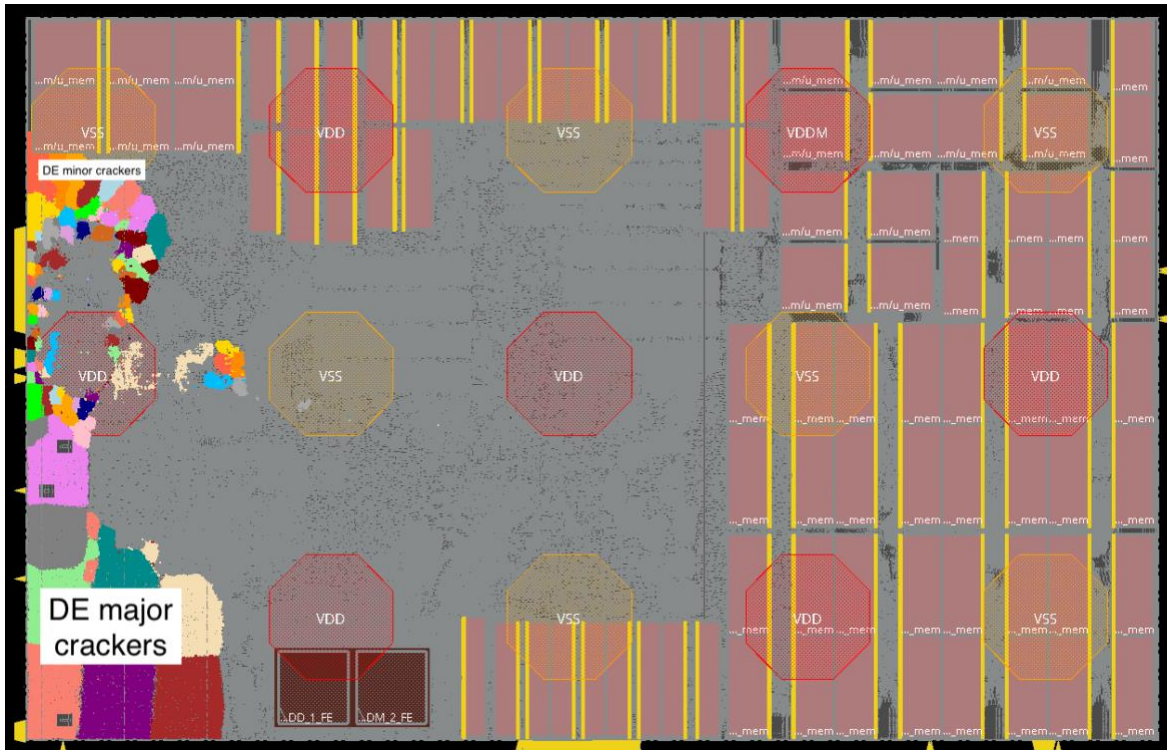


Figure 10 - DE module placement

tt_rv_mc

Midcore (MC) receives the dispatch information from the FE unit and assigns that dispatch information to their respective Int, Vec, and FP logic within the **map**, **db**, and **sch** modules. The scheduler (**tt_sch**) module tracks the various Ops dependencies. If an Op is ready to execute, then the respective Int, FP, or Vec Register file is read, and the intended arithmetic operation is performed. For a Load Store Op, the execution logic generates the address and sends this to Load Store unit. **tt_rv_mc** breaks down into two physical hierarchies (**tt_mc_mc1** and **tt_mc_mc2**) to enable quicker block turnaround time.

tt_mc_mc1

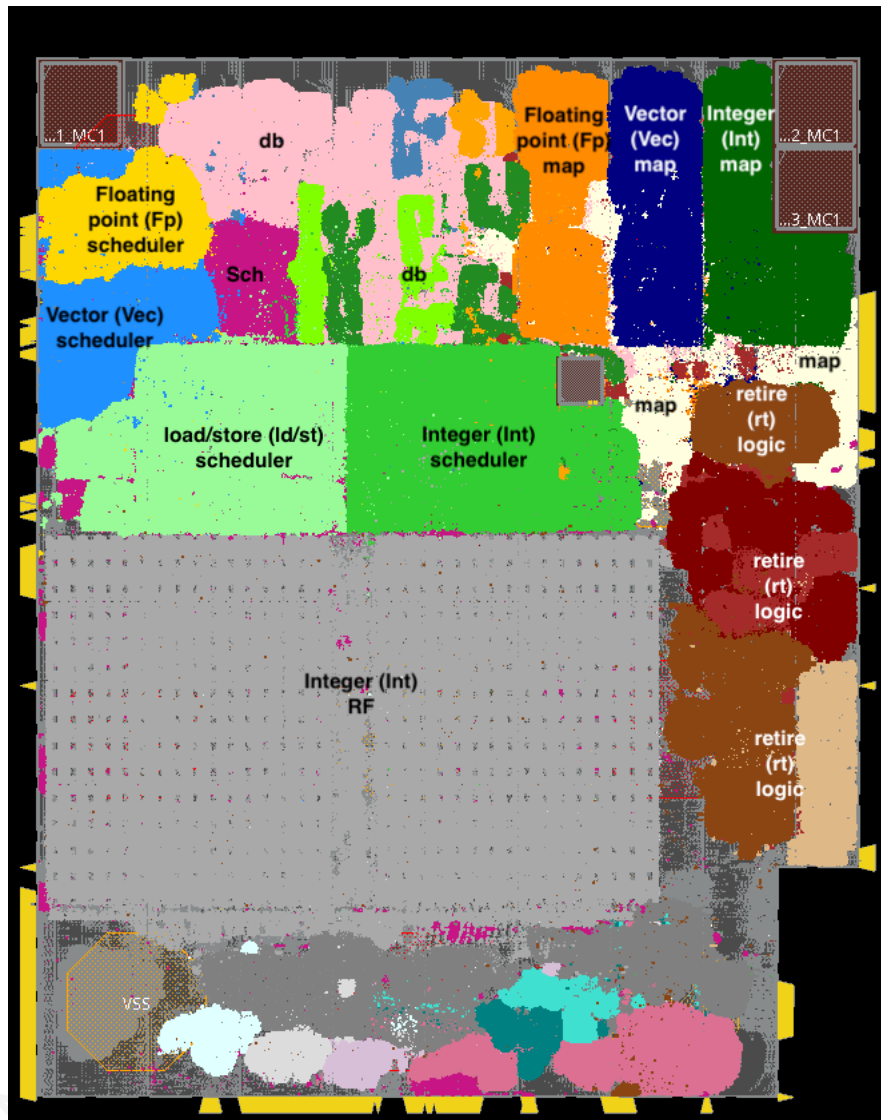


Figure 11 - tt_mc_mc1 module placement

The **tt_mc_mc1** block consists of:

- **Control logic:** **map**, **db**, **sch**, **cu1**, and **rt** modules.
- **Integer Register file:** Built out of latch arrays.
- **DataPath:** Execution logic.

tt_mc_mc2

The **tt_mc_mc2** block consists of floating point and vector register files along with their respective DataPath (execution logic):

RF Name	# of entries	Size/entry	# Read ports	# Write ports
Vector RF	160	256b	9	5
Floating Point RF	184	64b	11	9
Mask buffer	32	256b	5	2
VL	64	9	6	2

Table 8. tt_mc_mc2

Module Placement



Figure 12 - tt_mc_mc2 Module Placement

tt_rv_ms

Figure 13 shows the **tt_rv_ms** interface block between the core and the cluster level. This block contains muxing of debug units.

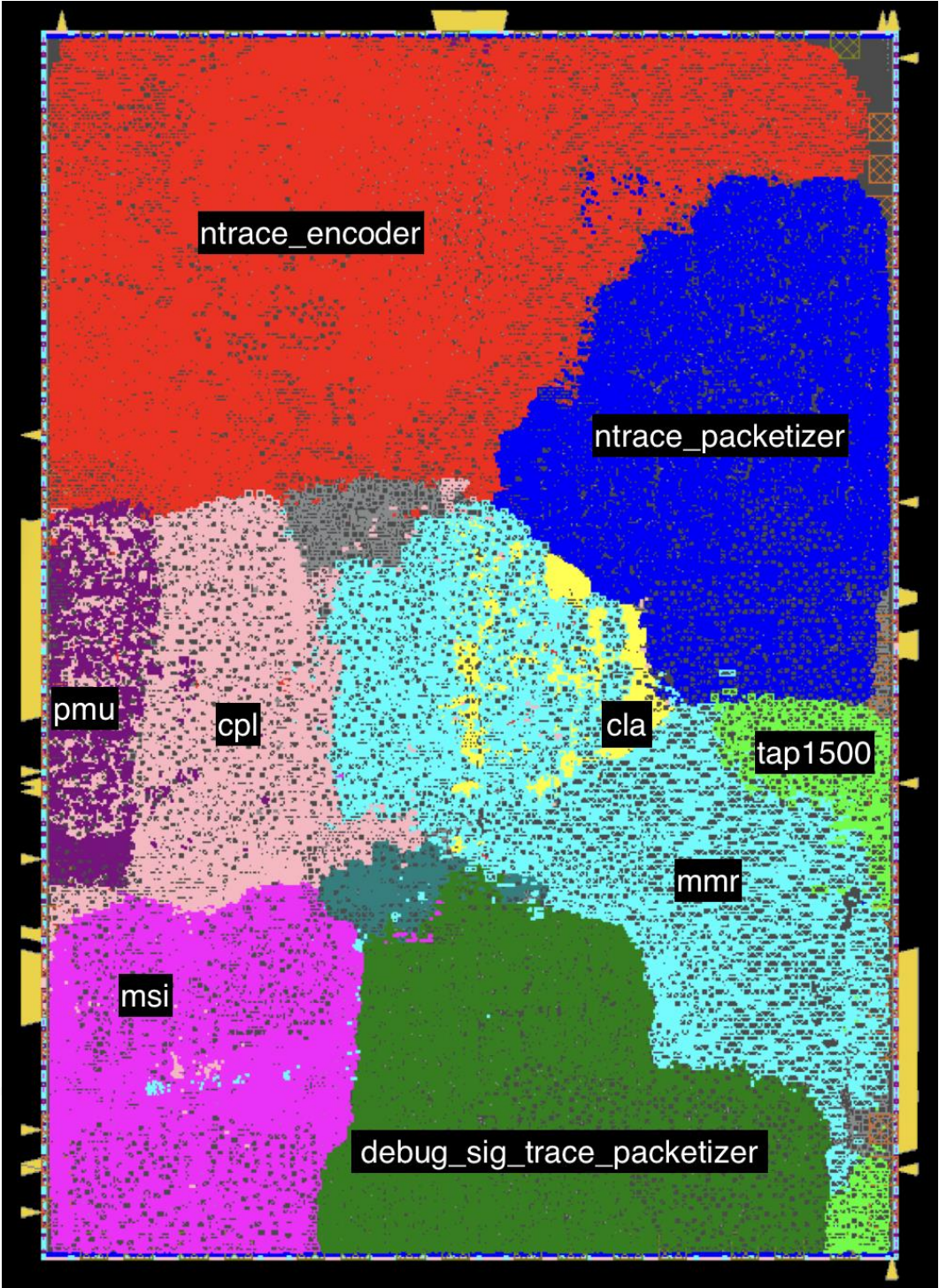


Figure 13 - tt_rv_ms module placement

Shared Cache

tt_sc_slice_top

The **tt-sc_slice_top** block is a macro-dominated block with both data and control macros. The width of the **tt_sc_slice_top** block is defined by the number of data macro columns used in the floorplan. Figure 14 shows the floorplan and pin placement for **tt_sc_slice_top**.

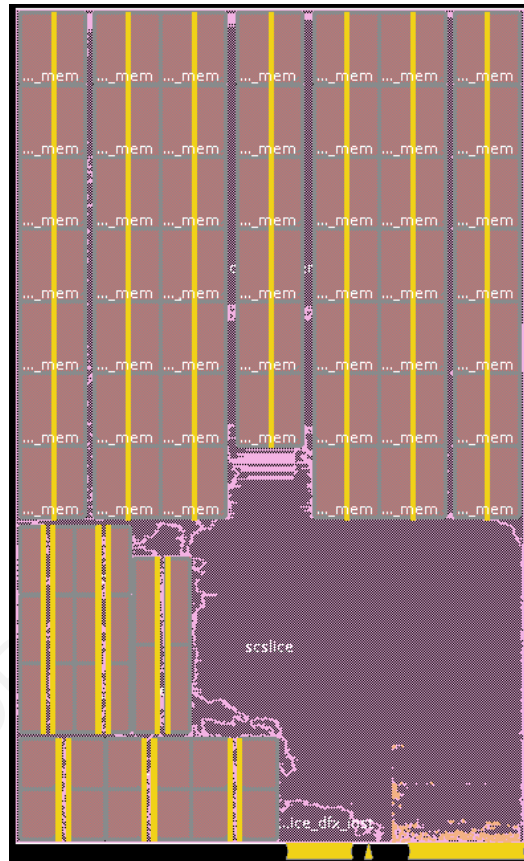


Figure 14 - tt_sc_slice_top floorplan

sc_slice_top

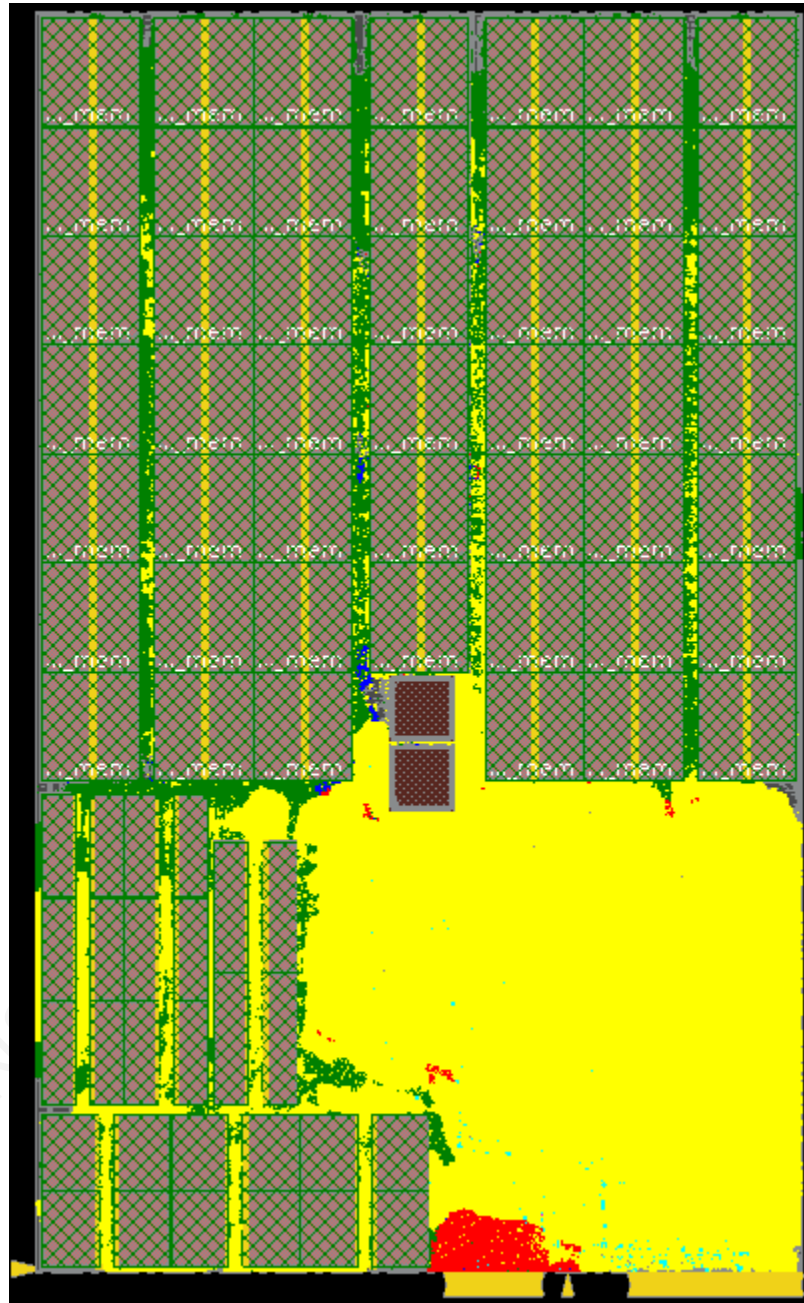


Figure 15 - tt_sc_slice_top module placement

The Shared Cache Pipeline consists of seven cycles:

- **Cycle 1:** The bypass into tag array from an incoming request or a ripe access from MSHR is set up to do the SC tag/state/replace and SNF tag/state array reads. The incoming request could be a Core-SC request, Fabric-SC snoop, or Internal SNF/SC cictim.
- **Cycle 2:** The tag/state/replace array reads are done.

- **Cycle 3:** The pipe access result is available, inline ECC detection and correction are performed, and the tag match is done on pipe reads. This information is then used to generate the hit/miss information and determine the hitway/vicway for both SC and SNF.
- **Cycle 4:** On pipe reads, the protocol planner uses the tag/state result and the request information from the MSHR to generate the MSHR plan to be executed. On a miss, the MSHR allocation logic arbitrates the SC/SNF victim.
- **Cycle 5:** The state/replace array writes, data array read/write starts, and the **Rdat_Ctl** first beat for read hits are presented to the Cross Bar, if possible.
- **Cycle 6:** The state/replace array and data array writes complete. Data muxing and ECC detection occur for data reads.
- **Cycle 7:** The data is sent to the MSHR data buffer and, if the **Rdat_Ctl** Bypass was successful, to the channel driver for **Rdat_dat**.

The **tt_sc_slice_top** block focuses on the critical paths from the core to the shared cache (**coretoscnw***). Tenstorrent recommends creating a bound close to the **NW_SCSLC*** and **SCSLC_NW*** ports to ensure minimal buffering on the Interface flops. These flops also have both a target insertion delay and Useful Skew turned off. Turning off Useful Skew for these IO groups (i.e., **reg2out** and **in2reg** interface flops that communicate with the staged flops at the **sc_partition** level) ensures no skew on these paths between interfaces. It also provides an average Insertion delay target for partition-level slicing. SC data macros are grouped together by DWs for placement. The signals from **harvest_strap/slice_num/sram_hold** have MCPs, while all Macro ReadOut paths are a 2-cycle MCP.

tt_sc_partition

tt_sc_partition instantiates 8 instances of **sc_slice_top** connected to all cores by an 8x9 crossbar with inputs coming from each core, and the extra NCIO output that connects to the Shared Cache Bridge (SCB) in the cluster. This block is highly floorplan dependent; running through physical synthesis will provide both a realistic instance count and better PPA optimization. Figure 16 shows the **sc_partition** floorplan with 8 instances of **sc_slice_top**. The crossbar (**xbar**) placement region is in the middle of **sc_partition** network channel. Pins related to the even cores are placed on the left edge. Pins related to the odd cores, fabric, and NCIO are placed on the right edge. **sc_slice_top** instances are placed in orientations such that their pins are at the shortest possible distance from the crossbar. Figure 17 shows the placement regions for some of the critical staging flops in the design.

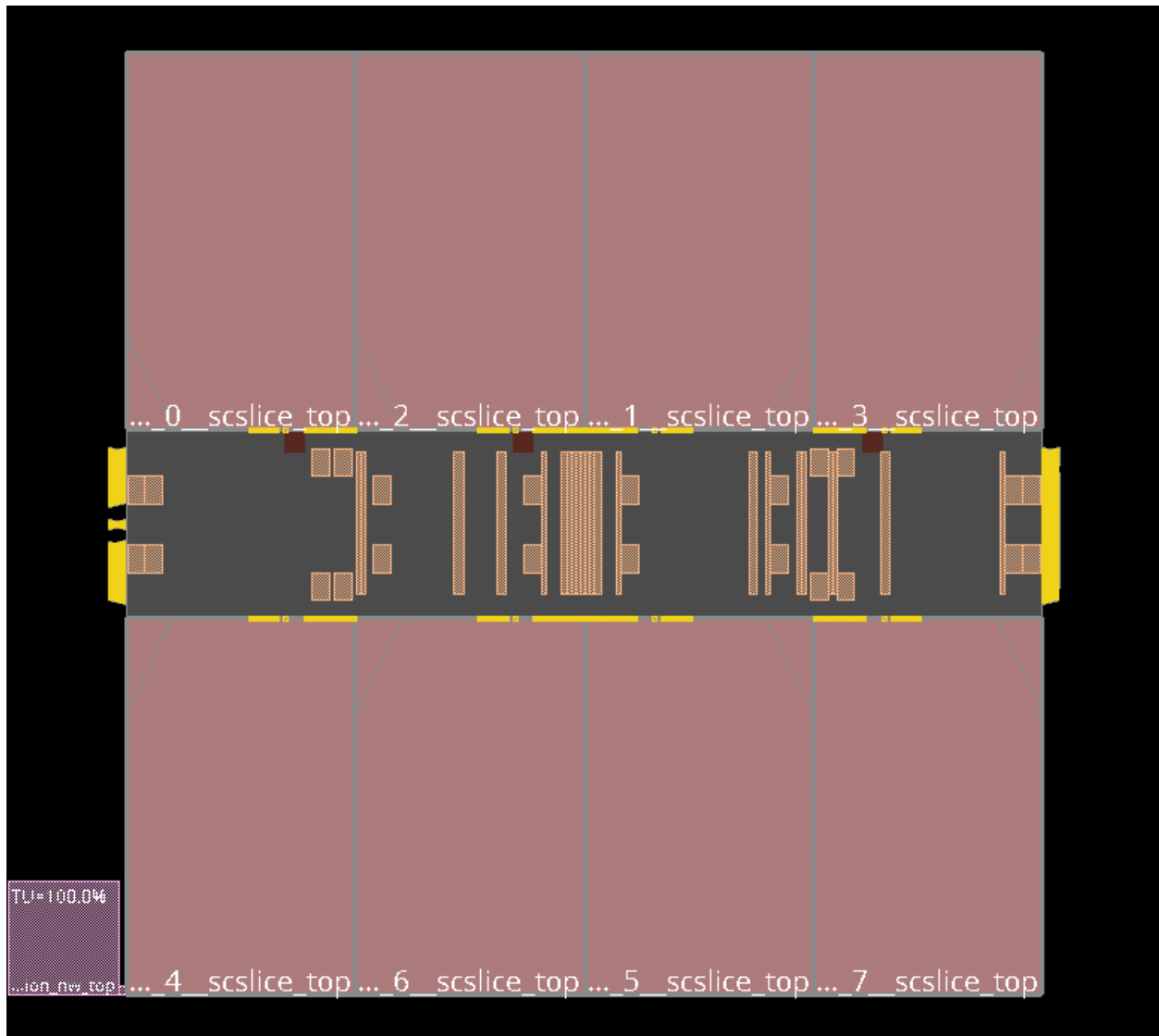


Figure 16 - sc_partition floorplan

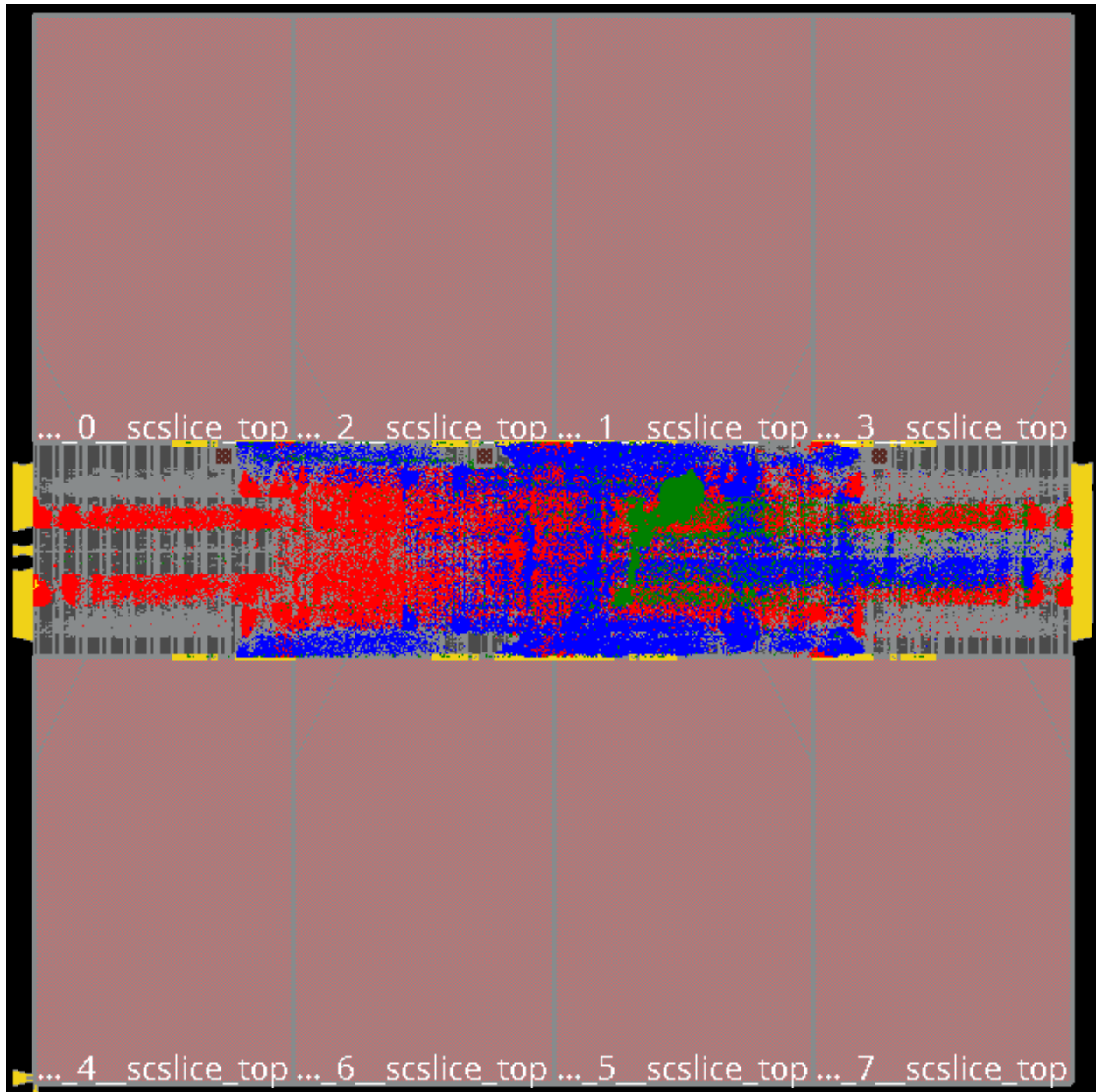


Figure 17 - tt_sc_partition module placement

sc_partition Recommendations

tt_sc_partition is a wire-dominated block with skid-buffer (R-S) stage flops to connect Core-SC (**coretoscnw**) and SC-Fabric (**sctofbnw**). The **sc_partition** block has an 8x9 crossbar between 8 cores and 9 destinations (8 slices and a fabric NCIO interface). The **sc_partition** block width is 4 times the width of each slice, and the height of the **sc_partition** block is 2 times the **sc_slice_top** height plus the height of a horizontal channel wide enough to fit either:

- 13 data buses (~442um)
- 8 CR-SC data buses
- 5 SC-FB data buses

The **tt_sc_slice_top** blocks are placed in the **tt_sc_partition** block such that the even slices are placed to the left and odd slices are placed to the right. The R-S staged flops are bound (placement regions) such that each of the R-S (**xbor*/xbos***) stage is optimally and equally spaced for the TX paths from cores to **sc_partition**, continuing to **sc_slice_top**. RX path R-S flop stages are equally spaced. Figure 18 illustrates the R-S stage placement in **sc_partition**, where:

- The left side of Figure 18 shows:
 - Core-SC stages with the TX stages in red and the RX stages in green. The CR2SC TX path starts from primary inputs from the left-right side of the channel and goes through two R-S stages before the crossbar (**PStage0** and 1) and one R-S stage after the crossbar (**SStage0**).
 - RX stages start from far slices and go through one R-S stage before the crossbar (**PStage0**) and one stage after the crossbar (**SStage0**). The rest of the RX path to Cores is at the **sc_main**/cluster level. Figure 18 only shows the far slices (0, 4, 3 and 7) from the crossbar because the other slices are not critical and don't need placement regions.
- The right side of Figure 18 shows:
 - FB-SC stages when SC2FB RX and FB2SC TX stages in red and the SC2FB TX and FB2SC RX stages in green. SC2FB TX and FB2SC RX start from far slices (0 and 4), go through one R-S stage (**PStage0**), muxes with other slice outputs, and leaves the **sc_partition** channel on the right edge through one R-S stage (**SStage0**). Rest of the path is in the **sc_main**/cluster right channel.
 - The SC2FB RX and FB2SC TX paths start from the right side of the channel, go through one R-S stage (**PStage0**), and demux to all slices through another R-S stage (**SStage0**). Paths to and from other slices are non-critical and don't require placement regions.

The Clock gaters of each R-stage are controlled by the same and previous S-stages. Thus, overall QOR improves by placing R-S stages alternately, with S-stages equally spaced between two R-stages and vice versa. There are some exceptions for stages restricted by block boundaries and crossbar. The S-stage should always be placed after the R-stage because the S-stage input comes from its R-stage. The RX path has an extra (spare) S-stage. The equivalent R-stage is inside the slice. This extra spare S-stage helps with equidistant placement of R-S stages. The fabric bounds are also placed equidistantly.

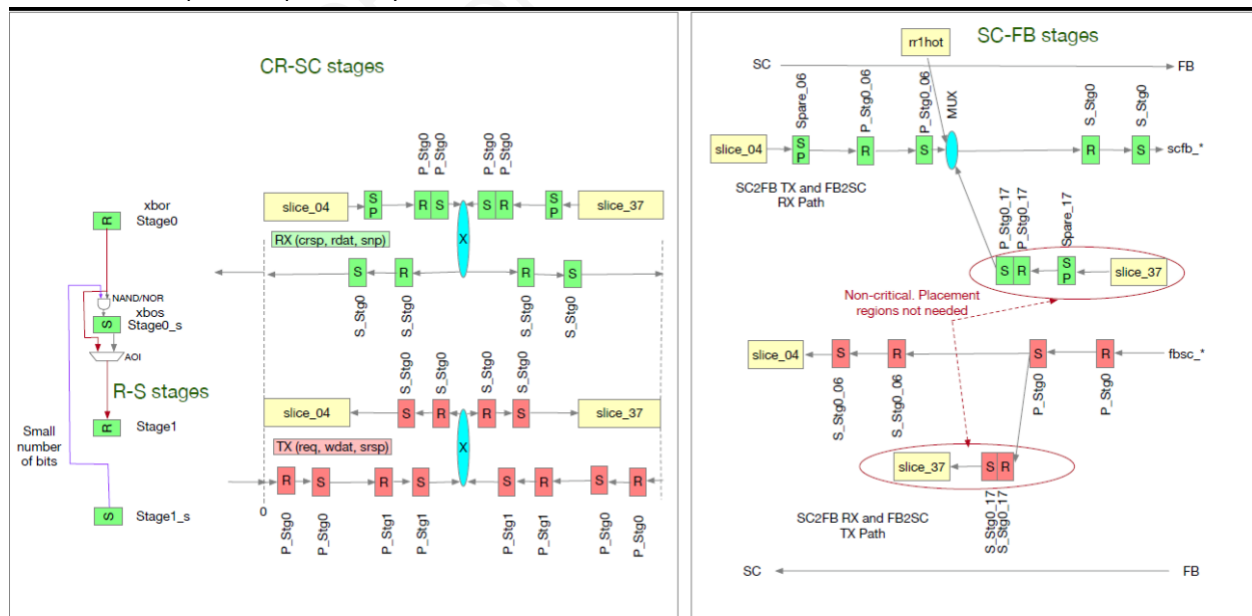


Figure 18 - **sc_partition** flop stages

Cluster

The cluster integrates 8 Cores and 2 SC partitions. Tenstorrent recommends flattening the **sc_main** hierarchy and running PNR for it at the cluster level. Top level cluster logic is placed above the SC partition and can access the cores/caches through the two vertical routing channels. External access is through ports placed above the right channel. Two additional channels are placed above and below **sc_partition_1** to provide routing between the two vertical channels. Figure 19 shows the cluster floorplan.



Figure 19 - tt_cluster floorplan

Cluster logic placement

The **sc0** hierarchy contains both **sc_partition** instances and bus connectivity for the SC/CPU and SC/Fabric communication. The **sc_partitions** are built as two identical hard macros with the remaining **sc0** logic placed at the cluster level.

SC/CPU and SC/fabric connections are done through the two vertical channels that make up most of the channel routing. Additional CPU and SC connectivity is also done in the vertical channels for the **cl_ring**, **trace**, **pmnw**, and **cla** networks.

sc0 hierarchy

The **sc0** hierarchy connects the core and **SharedCache** (SC) through skid-buffer (R-S) stage flops like **sc_partition**. This hierarchy also includes SC-FB skid-buffer stage flops to connect with the Fabric interface. Async FIFOs are placed between the two **sc_partitions** and the SC Bridge (SCB) because these run at different frequencies. Figure 21 shows added placement regions for R-S stages that manage routing/timing of these wire-dominated channels. **Figure 21 show** asc cluster floorplan.

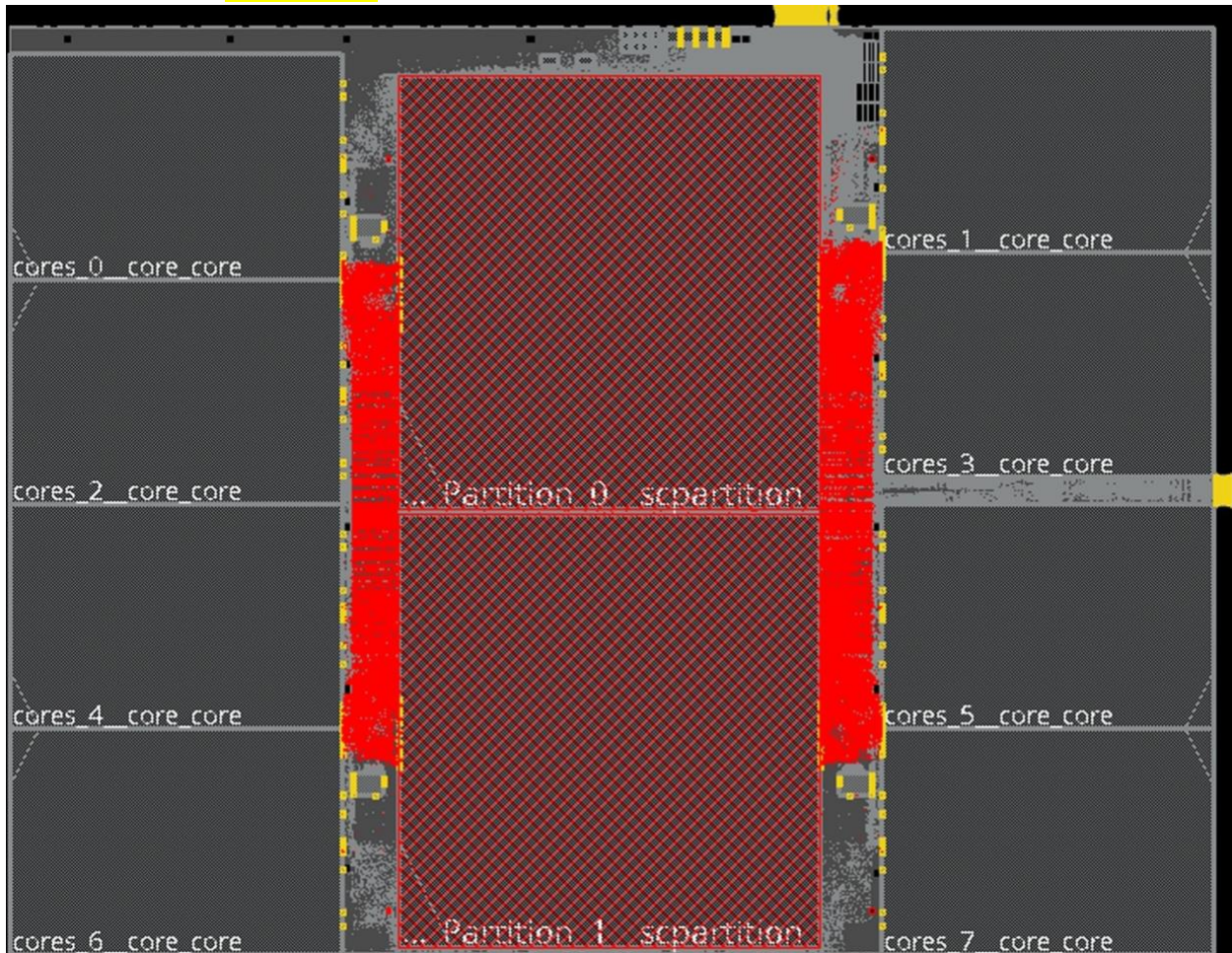


Figure 20 - asc_cluster



Figure 21 - tt_cluster placement region

Ring Networks

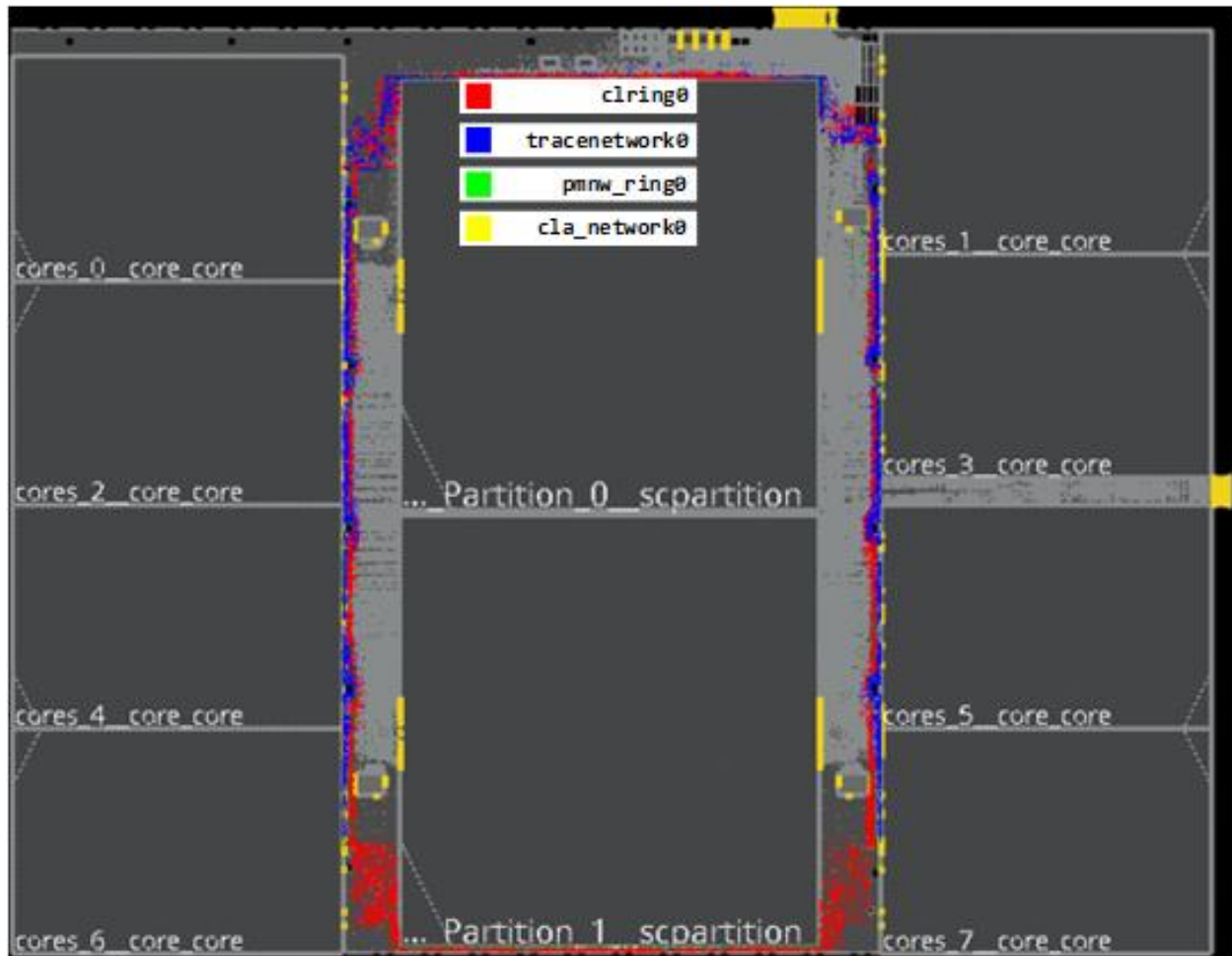


Figure 22 - Ring Network

SC/Fabric connectivity is completed through three additional logic hierarchies. The **chi_cdc_wrapper** async FIFO connects to the sc0 to convert the high speed **CLCLK** buses to the **FBCLK** domain. This is followed by a staging flop hierarchy (**cluster_ft0**) and SCB logic (**scb_top_wrapper0**), and then connected to the Cluster Ios.

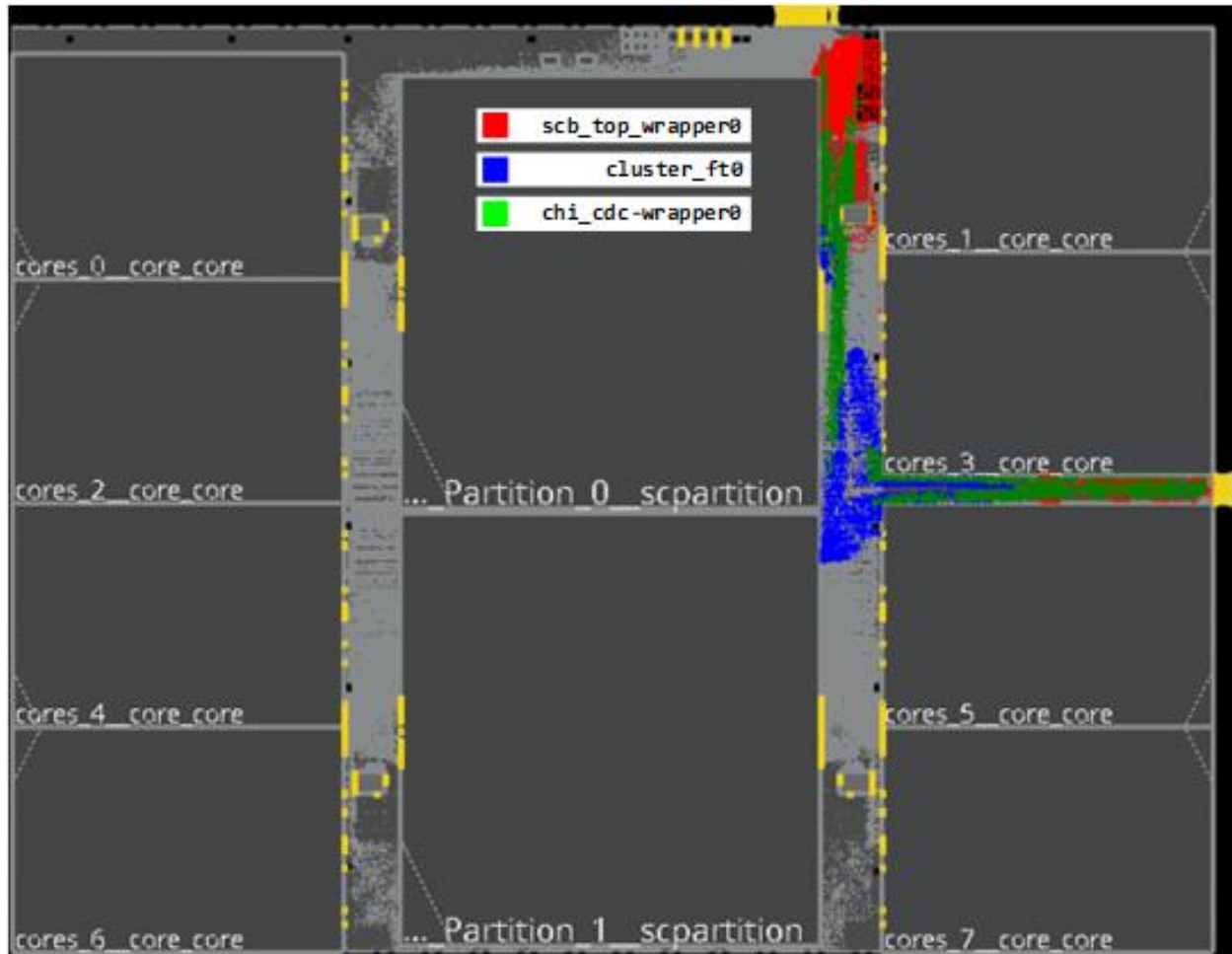


Figure 23 - tt_cluster module placement

The remaining cluster level logic is placed above the caches. The SCB module is anchored just below the cluster IO ports. PLLs are centered in the middle of the top channel with the trace macros placed to the left and CPL macros placed to the right. The placement tool automatically places the remaining logic.

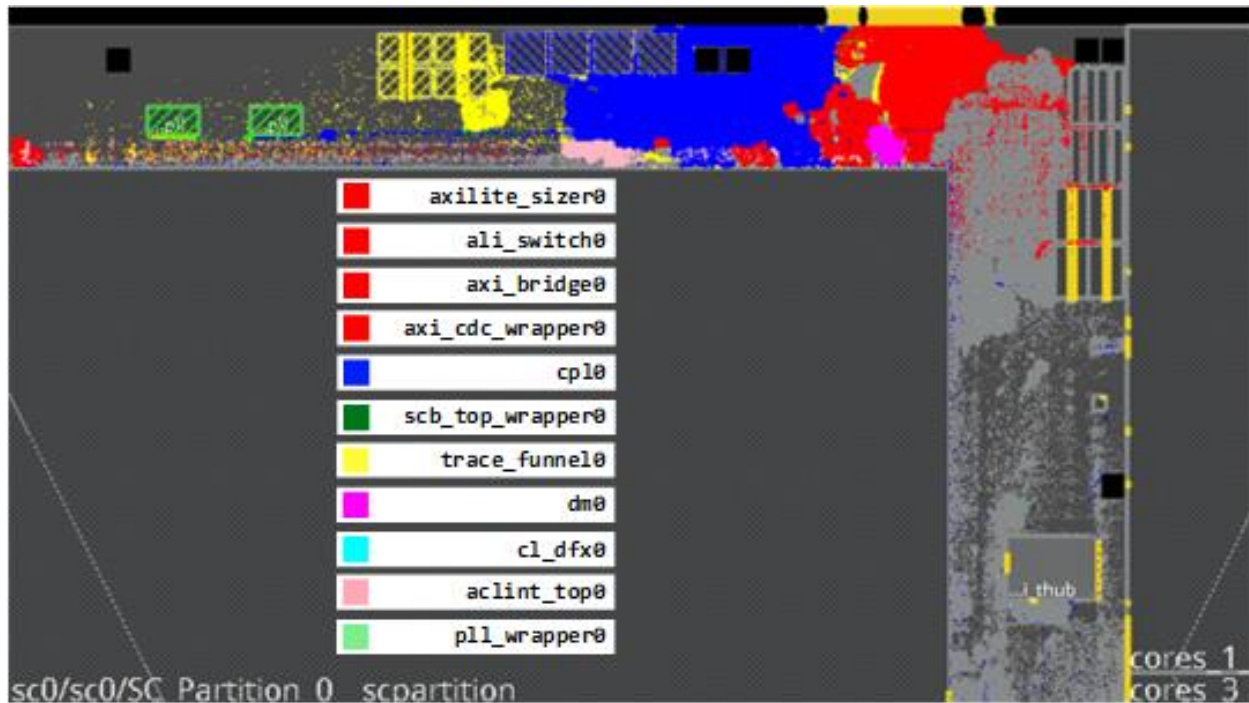


Figure 24 - asc_cluster module placement

Clock Distribution

CLCLK is the high-speed clock generated by the PLLs internal to the cluster and used for the CPUs, Caches, and logic connected to these modules. It spans the vertical channels. Tenstorrent recommends implementing through the **HTree** construction.

Table 8 lists the additional clocks provided by external inputs to the Cluster. You implement these through normal CTS.

Clock	Frequency	Source	Connectivity
CLCLK	Technology dependent	Internal PLL	CPUs, SC, trace, CL network, PM network
FBCLK	Technology dependent	External	SCB, AXI, debug
SOCCLK	400MHz fixed	External	CPL
REFCLK	100MHz fixed	External	PLLs, ACLINT
TCK	200MHz fixed	External	DFX, JTAG

Table 9. tt_cluster clocks